# The Sub-Floor Coarsest Level

A starved level 0 manufactures a permanent runt, not a split cell — and "`LINE SEARCH STALLED`" was never a failure signal

August 2026


**Abstract**

Four attempts to make the $N$=300 Phase 1 ladder cheaper have failed. This report tests the complementary hypothesis — that the *ladder itself* is the defect, because three of its five levels sit below the ∼250–300 vertices-per-cell floor of `docs/reference/winner_take_all_partition_gap.md` §4b. The test reproduces $N$=300's level-0 condition at $N$=100, where it is affordable and the control is a validated deliverable: the same $\lambda$, seed, init and *identical* finest mesh, with only the base mesh dropped from 100×96 (96 v/cell) to 60×52 (31 v/cell). Three results. (1) The pre-registered prediction is **refuted**: **zero** fragmented cells at every level, final included — a starved coarsest level does not manufacture disconnection at $N$=100, so $N$=300's persistent fragmentation of cells 274 and 290 has another cause. (2) It does manufacture a *permanent* area runt: the worst cell opens at 46.80% off target on the 31 v/cell level and is still **15.92**% on the finest mesh — 37× more vertices, never crossing the 5% gate — against a control that ends at 0.78%. Damage done at level 0 is inherited, not healed. (3) A methodological correction that invalidates part of the reasoning that motivated the test: the `LINE SEARCH STALLED` warning is **not** a failure signature. The validated control floors its Armijo step on four of five levels and then runs exactly `refine_patience` = 30 more iterations before the trigger fires. Flooring *is* normal termination; only its *timing* is diagnostic — 31 v/cell floors at iteration 31, 96 v/cell never floors across 30,000.


**Status — measured; hypothesis REFUTED, with a methodological correction**

Stage 1 ran to completion and reads out cleanly. The pre-registered prediction (fragmented cells the control did not have) did **not** occur, so by its own gate **Stage 2 must not be run**: `parameters/torus_300part_above_floor_ladder.yaml` remains unrun. The secondary finding — a permanent 15.92% runt — is a positive result on the *imbalance* axis. It is related to but distinct from P3 of `docs/plans/PHASE1_N1000_VALIDITY_PLAN.md`: the threshold crossed here is the ∼90 v/cell boundary at which a level stops running at all, not P3's ∼250–300 v/cell gate floor — a distinction worth 3× in mesh budget at $N$=1000, and set out in §6. The stall-guard correction in §5 supersedes the framing in commits `1b2ad04` and `db737ea` and is fixed in code (§5.1).

**Provenance**

| | |
|---|---|
| Date | 2026-08-14 (levels 0–3 overnight; level 4 re-run after a machine power fault interrupted it, see below) |
| Surface | torus $T(R{=}1.0,\ r{=}0.6)$, $N{=}100$, $\lambda{=}5.1$, seed 84172851, seeded init, 5 levels |
| Control | `results/run_20260709_081548_surftorus_npart100_..._lam5.1_seed84172851` — the validated $N{=}100$ deliverable: 0 dead / 0 weak / 0 imbalanced / 0 fragmented, worst cell 0.78%, min peak density 0.99994, initial (contour-extraction) perimeter 214.3413, Phase 1 wall 48,131.8 s. Ladder $100{\times}96 \to 348{\times}328$, increments $+62/+58$. |
| Sub-floor arm | config `parameters/torus_100part_subfloor_ladder.yaml` — identical in every relaxation knob except the base mesh $60{\times}52$ (increments $+72/+69$), so the ladder still terminates at the *same* finest mesh $348{\times}328$, $V{=}114{,}144$. `struct_trigger_enabled: true`. |
| Runs | `results/run_20260814_002405_...` (levels 0–3; level 4 interrupted at iteration ∼1,076 by an undervoltage hard reset of the host, unrelated to the code) and `results/run_20260814_091130_...` (level 4, resumed from `solution/checkpoint_level04.h5` via `--resume-from`; 1,100 iterations, 12,490 s). |
| Scripts | `docs/experiments/06-subfloor-ladder/make_figures.py` (figures); `testing/watch_level_gates.py` (per-level gates, which copies each checkpoint before the pipeline overwrites it); `testing/check_fragmentation.py` (final gates). |
| Versions | Python 3.12.13, numpy 2.4.4, scipy 1.17.1, h5py 3.16.0; macOS 15.3.1 arm64 (Mac mini). |
| Anchors | sub-floor level 0 floors at iteration **31** and ends at 60; every floored level in both runs ends with exactly **30** iterations at step $9.0949470177292824 \times 10^{-13}$; final worst cell 32 at 15.92% (abs 0.03771); sub-floor initial perimeter 214.1032. |

# Contents


**1 The question** **4**

**2 Method** **4**
- 2.1 Reproduce the condition where it is affordable . . . 4
- 2.2 Pre-registration . . . 4
- 2.3 Per-level instrumentation . . . 4

**3 Result 1 — the prediction is refuted** **5**

**4 Result 2 — what a starved level 0 does cost** **5**

**5 Result 3 — "LINE SEARCH STALLED" is normal convergence** **6**
- 5.1 The fix . . . 7

**6 What this means for scaling** **8**
- 6.1 Which threshold this report measured . . . 8
- 6.2 Cross-check at $N{=}300$ . . . 8
- 6.3 What does not follow . . . 9

**7 Conclusions** **9**

**Related documents** **10**

# 1 The question

At $N$=300 the Phase 1 ladder inherited from $N$=100 — base 100×96, increments $+62/+58$ — gives the coarsest level **32** vertices per cell, where at $N$=100 the same schedule gives 96. Three of its five levels sit below the $\sim$250–300 v/cell floor established in §4b of the gap document. Meanwhile cells 274 and 290 are fragmented in *every* $N$=300 run on record — 3-level, 5-level, and adaptive — always the same two cells in the same place, never healed by any downstream level; and cell 299 is always the worst runt.

Four arms (territory-aware relaxation, the soft area constraint A2, its exact-at-level-0 hybrid, and the adaptive within-level switch) have tried to make that ladder *cheaper*. None has asked whether the ladder is *wrong*. The question here is:

> Does a coarsest level below the resolution floor manufacture the defects we see at $N$=300, which every later level then inherits?

# 2 Method

## 2.1 Reproduce the condition where it is affordable

Testing this at $N$=300 costs multiple days per arm. Instead the sub-floor condition is reproduced at $N$=100, where a matched control already exists and is known clean. The arm changes exactly one thing: the base mesh drops from 100×96 to 60×52, giving 31 v/cell to match $N$=300's 32. The increments are re-chosen ($+72/+69$) so the ladder still ends at the *same* finest mesh, 348×328, $V$=114,144. Everything else — $\lambda$=5.1, seed 84172851, seeded init, 5 levels, every trigger and tolerance — is the control's.

## 2.2 Pre-registration

Written into the config header before the run:

- **Confirms** if the run produces fragmented cells the control did not have, and the guard reports level 0 dying early.
- **Refutes** if it comes back with 0 fragmented — in which case $N$=300's fragmentation has another cause, "equally worth knowing, for one $N$=100 run instead of four days at $N$=300."
- **Confound to report separately**: at 31 v/cell the interface may be unresolvable, producing dormant or weak cells. That is a *distinct* failure from fragmentation and must not be conflated with it.

Stage 2 (`parameters/torus_300part_above_floor_ladder.yaml`, which deletes the three sub-floor levels at $N$=300) was written at the same time and explicitly gated: *do not run until Stage 1 reads out.*

## 2.3 Per-level instrumentation

`run_relaxation` evaluates the three validity gates only on the *final* solution, and the pipeline keeps only the newest per-level checkpoint, deleting all of them once the run finishes. The per-level history is exactly what answers *where* damage first appears, so `testing/watch_level_gates.py` was written for this run: it polls the run directory, copies each `checkpoint_level*.h5` aside before it can be replaced, and runs all three gates on it.

## 3 Result 1 — the prediction is refuted

**Zero fragmented cells at every level, including the final solution**, with not even a sub-threshold speckle component (`cells with >1 raw component: 0`).

| after level | $V$ | v/cell | dead/weak | min peak | imbalanced (worst) | fragmented |
|---|---|---|---|---|---|---|
| 1 | 3,120 | 31 | 0 / 0 | 0.7367 | 74 (46.80%) | **0** |
| 2 | 15,972 | 160 | 0 / 0 | 0.9978 | 1 (18.78%) | **0** |
| 3 | 38,760 | 388 | 0 / 0 | 0.9980 | 1 (18.13%) | **0** |
| 4 | 71,484 | 715 | 0 / 0 | 0.9981 | 1 (16.59%) | **0** |
| 5 (final) | 114,144 | 1,141 | 0 / 0 | 0.9982 | 1 (15.92%) | **0** |
| *control, final* | 114,144 | 1,141 | 0 / 0 | 0.99994 | **0** (0.78%) | 0 |

So a coarsest level at 31 v/cell — three times starved relative to the control, matching $N$=300's condition — does *not* manufacture disconnection. Per its own gate, **Stage 2 is not to be run**, and the origin of $N$=300's cells 274 and 290 remains open.

The pre-registered confound did appear and stayed distinguishable: the 31 v/cell level ends with min peak density 0.7367 against ∼0.998 everywhere downstream — an interface the mesh can barely resolve — but no cell is dead or weak, so the dormant gate and the connectivity gate never had to be disentangled.

## 4 Result 2 — what a starved level 0 does cost

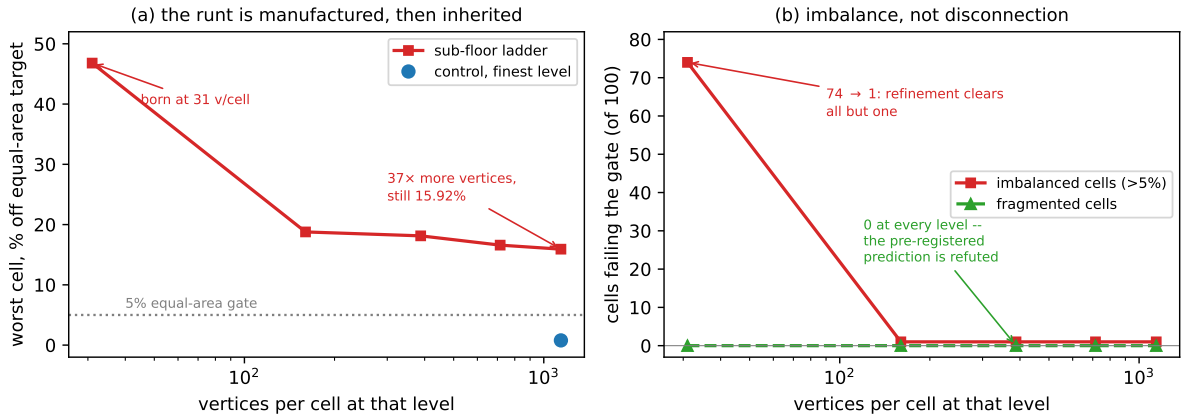


Figure 1: The damage a sub-floor coarsest level leaves behind. (a) The worst cell's discrete-area deviation opens at 46.80% on the 31 v/cell level and is still 15.92% on the finest mesh, never crossing the 5% gate, against a control that ends at 0.78% on the identical mesh. (b) Refinement clears 73 of the 74 imbalanced cells but not the last one; fragmented cells are 0 at every level.

The arm is a *gate failure*: 1 of 100 cells over the 5% threshold, cell 32 at 15.92% (absolute 0.03771, which is exactly the Phase 2 iteration-0 equal-area constraint violation this solution would hand downstream).

The trajectory is the finding. Between level 1 and level 5 the mesh grows 37× — from 3,120 to 114,144 vertices, ending at the control's own finest mesh — and the worst cell improves only 46.80% → 15.92%. It is still three times over the gate. **A defect created on a starved coarsest level is inherited, not healed**, even by a mesh that on the control's ladder produced 0.78%.

**One confound, stated.** The arm sets `struct_trigger_enabled: true` and the control does not, so the two differ in a second knob. It bit level 1 only, stopping it at 3,000 iterations ("structure frozen: 3 label flips over the last 2,000"); every other level ended on the ordinary plateau trigger. This cannot touch Result 1 — a trigger that stops a level early cannot *remove* fragmentation that a longer run would have shown, and there was none at any level — but it means the 15.92% is a mild *upper* bound on how well this ladder could do: an un-triggered level 1 might have ground the runt down further. It would have to recover 11% to reach the gate. The re-based $N$=300 follow-up (`parameters/torus_300part_rebased_ladder.yaml`) leaves the trigger off for exactly this reason.

Two controls on that reading. First, the geometry at large is comparable: the initial contour-extraction perimeter is 214.1032 here against the control's 214.3413 — 0.11% apart — so the harm is concentrated in one cell's area share, not distributed. Second, the sub-floor ladder is *cheaper*, not more expensive: $\sim$24,300 s against the control's 48,132 s, because the control's level 0 alone burns 19,583 s running to the 30,000-iteration cap. Halving the ladder cost while breaking the partition is precisely the trade the four prior arms were making, arrived at from the opposite direction.

## 5 Result 3 — "LINE SEARCH STALLED" is normal convergence

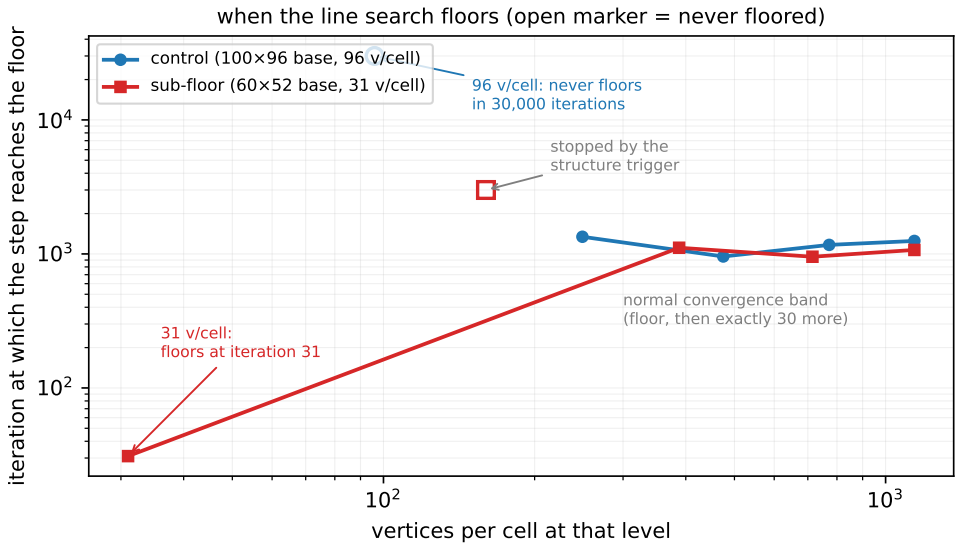


Figure 2: When the Armijo step reaches the backtracking floor $9.0949 \times 10^{-13} = \text{step}_0\rho^{40}$, against vertices per cell. Filled markers floored; open markers never did. Every floored level — in the *validated control* as much as in the arm — then runs exactly `refine_patience` = 30 more iterations before the refinement trigger fires.

The stall guard added in `1b2ad04` reports a floored line search as "a dead optimizer masquerading as convergence," and `db737ea` built on it: the claim that $N$=300's two coarsest levels "provably do not converge — they stall at iterations 31 and 41." Checking the guard against the control shows this reading is wrong.

| | v/cell | iterations | first floored | trailing floored block | |
|---|---|---|---|---|---|
| *control* `run_20260709_081548` — 0 imbalanced, 0 fragmented | | | | | |
| level 0 | 96 | 30,000 (cap) | — | 0 | never floors |
| level 1 | 249 | 1,372 | 1,343 | **30** | |
| level 2 | 475 | 986 | 957 | **30** | |
| level 3 | 772 | 1,197 | 1,168 | **30** | |
| level 4 | 1,141 | 1,279 | 1,250 | **30** | |
| *sub-floor arm* | | | | | |
| level 0 | 31 | 60 | **31** | **30** | |
| level 1 | 160 | 3,000 | — | 0 | structure trigger |
| level 2 | 388 | 1,141 | 1,112 | **30** | |
| level 3 | 715 | 982 | 953 | **30** | |
| level 4 | 1,141 | 1,100 | 1,071 | **30** | |

The trailing block is **exactly 30** in every floored level of both runs, and `refine_patience: 30`. The mechanism is plain: PGD reaches an iterate where no backtracked step satisfies Armijo, the step floors, the energy freezes, and 30 iterations later the plateau trigger's patience is satisfied and refinement fires. *That is how a healthy level terminates.* The control's level 4 ends frozen at $E$=92.155173, $\|g\|$=3.1609; the arm's ends at $E$=94.160421, $\|g\|$=3.1740. Same behaviour, and one of them is the validated deliverable.

Three consequences.

1. **The guard fires on every healthy level termination.** `_warn_if_stalled` triggers when any step was rejected in the run-up to the trigger — but the patience window *is* 30 rejected iterations by construction, so the condition holds identically on good and bad runs. It was calibrated on the A2 arm, where flooring genuinely was pathological (`docs/experiments/05-soft-area-constraint/` §Result 3), but the terminal signature is the same on the exact path, so it cannot discriminate.
2. **Only the *timing* of the floor is diagnostic, and it is strongly resolution-dependent.** Figure 2: a cold level 0 at 31 v/cell floors at iteration **31**; the same cold level 0 at 96 v/cell never floors across 30,000 iterations; warm-started levels at $\geq$388 v/cell floor at $\sim$950–1,250. The $N$=300 level-0 figure this experiment set out to explain — "stalls at 31" at 32 v/cell — is reproduced here almost exactly at 31 v/cell. That reproduction stands; the interpretation of it as "does not converge" does not.
3. **Stage 2's refutation criterion was unusable as written.** It said a stall *above* the floor would refute the hypothesis — but the control stalls above the floor on four levels, as does this arm's own level 4 at 1,141 v/cell.

### 5.1 The fix

The guard is retained but re-based on *onset*: it fires only when the step reaches the floor inside the first `_STALL_EARLY_ITERS` (200) iterations of a level, and is silent otherwise, because flooding later is convergence. A ratio of the level's own length would not work — a starved level floors early *and* terminates early, so 31 of 60 iterations is 51.7%, not a small fraction — so the test is against an absolute iteration count. The earliest healthy floor on record is control level 1 at iteration 1,343, leaving a 6.7× margin. The message is renamed from `LINE SEARCH STALLED` to `UNDER-RESOLVED LEVEL` and now says what to do about it.

#### Replay validation

The rule was replayed offline over the STEP column of *every* PGD summary trace in `results/` — 130 levels across 40 runs, $N$=10 to $N$=300. Of these, 24 fire and 106 are silent (24 of the silent

never floored at all). The threshold is not a tuned parameter: **it falls in an empty gap.**

| | levels | first floored iteration |
|---|---|---|
| fires (onset $< 200$) | 24 | 18 – **100** |
| *(no level anywhere in* `results/` *lands here)* | 0 | 101 – 313 |
| silent, floored later | 82 | **314** – 22,574 |
| silent, never floored | 24 | — |

A level's line search either dies within its first 100 iterations or survives past 314; nothing in three months of runs sits between. Anything from 101 to 313 would give the same verdict on every level measured.

All five levels of the validated control are silent, including its level 0 at 96 v/cell. Every level below $\sim$90 v/cell that floors at all fires: $N$=300's levels 0 and 1, $N$=200's level 0 (48 v/cell, floor at 21–27), $N$=150's level 0 (64 v/cell, floor at 47), and this arm's level 0.

Two things the replay shows that the resolution framing alone does not. **Low resolution is the usual cause but not the only one**: the known-bad $N$=100 $\lambda$=2.1 run `run_20260701_143238` fires at 193 and 501 v/cell, correctly — those levels did die at iterations 47 and 100, and that config is documented as outside the $\lambda$ working window. So the guard reads "this level died before doing any work," with resolution the first thing to check and `lambda_penalty` the second; the warning text says both. **And it catches the adaptive soft-area arm**: `run_20260813_003231` level 3, at 257 v/cell — above the floor — floors at iteration 23, which is the soft treatment's line-search collapse of report 05, not a resolution problem.

Finally, an honest limit: this rule does *not* detect the A2 pathology where every level floored *late* and the collapse was still real. Late collapse needs a different signal, and none is proposed here.

# 6 What this means for scaling

## 6.1 Which threshold this report measured

It is *not* the $\sim$250–300 v/cell *gate* floor of `docs/reference/winner_take_all_partition_gap.md` §4b — the resolution at which a level can itself drive the worst cell under the 5% gate. What the sub-floor arm crossed is a much lower and much sharper boundary: whether the level *runs at all*. From the replay of §5, by vertices per cell:

| v/cell | iterations before the step floors | |
|---|---|---|
| 31, 32, 48, 64, 83 | 18–47 | **dies** |
| 92, 96 | 1,141, or never in 30,000 | **runs** |

The boundary sits between 83 and 92 v/cell, an order of magnitude below the gate floor. The mechanism is then not "a coarse level does bad work" but "a coarse level does *no* work": it dies in tens of iterations, the cheap equalization phase never happens there, and the first level that does run starts from a badly imbalanced field instead of from a nearly balanced one. That is exactly the arm measured here — the control's level 0 at 96 v/cell runs 30,000 iterations and equalizes from the seeded init; the arm's at 31 v/cell dies at iteration 31.

## 6.2 Cross-check at $N$=300

The same quantities were reconstructed for free from the $\lambda$=11.5 $N$=300 control's saved trace iterates (its `traces/*_internal_data.hdf5` store `x`; rebuild each level's mesh and run the gates):

| stage | mesh | v/cell | imbalanced | worst % |
|---|---|---|---|---|
| seeded init | — | — | 230 | 44.02 |
| after L0 | 100×96 | 32 | 234 | 45.57 |
| after L1 | 162×154 | 83 | 234 | 49.26 |
| after L2 | 224×212 | 158 | 10 | 36.15 |
| after L3 | 286×270 | 257 | 4 | 27.04 |
| after L4 | 348×328 | 380 | 3 | 24.81 |

Both of $N$=300's coarse levels are below the boundary and both die, handing forward what the init gave them (230 → 234). The equalization therefore happens at level 2 (158 v/cell), which *converges* there — worst % flat at 36.15 over its last 1,000 iterations — and levels 3–4 then improve with decelerating returns (−9.11, −2.23 points). So the same mechanism this report measures at $N$=100 is visibly present at $N$=300.

### 6.3 What does not follow

Two limits on how far to push this, both important.

**The floor is cheap, not expensive.** An earlier draft of this section put the fix at $V \propto N$ with ∼250–300 v/cell at every level, which at $N$=1000 would demand a *coarsest* level of $V \approx$ 250–300k. That inherited the wrong threshold. At the measured ∼90 v/cell boundary the coarsest level at $N$=1000 is $V \approx$ 90k — a factor of 3 smaller, and unremarkable next to the finest meshes already run here.

**A dying level 0 is not by itself fatal.** $N$=200 reaches 0 imbalanced (worst 1.65%) with a level 0 at 48 v/cell that also dies at iteration 21 — because its *level 1*, at 124 v/cell, runs 12,155 iterations and does the equalization there. The requirement that fits every run measured is therefore weaker and less tidy than "level 0 must clear a threshold": *some early, cheap level must run long enough to equalize before the expensive levels begin.* $N$=100 gets one at level 0, $N$=200 at level 1, and $N$=300's first is level 2 at 158 v/cell running 6,658 iterations — later, and less than half as long.

Whether giving $N$=300 an earlier, longer-running equalization level would move its converged answer is **untested**. This report shows the converged point *is* start-dependent — identical finest mesh, 0.78% vs 15.92% — which makes the question worth asking; it does not answer it. Note the naive version of that test is not the one to run: `parameters/torus_300part_rebased_ladder.yaml` lifts level 0 only to 51 v/cell, still inside the dying regime.

## 7 Conclusions

1. **The hypothesis is refuted on its stated terms.** A sub-floor coarsest level does not manufacture disconnected cells at $N$=100: 0 fragmented at all five levels. $N$=300's cells 274 and 290 have another cause, and Stage 2 should not be run. This cost 6.8 h instead of the several days a direct $N$=300 test would have.
2. **It is confirmed on an axis it did not pre-register.** A starved level 0 manufactures a runt that survives 37× refinement onto the control's own finest mesh: 46.80% → 15.92%, still three times over the gate, against 0.78%. Damage at level 0 is inherited.
3. **A pre-registered experiment can be refuted and still pay.** The fragmentation prediction failed; the run nonetheless produced the first controlled measurement of what the resolution floor is worth, and it did so by changing exactly one variable against a known-good control.

4. **Check a new diagnostic against a known-good run before building on it.** The stall guard was calibrated on a failing arm and never run against the validated control, where it fires on four of five levels. Two commits reasoned from it. A guard that has only ever seen sick patients cannot tell you what healthy looks like.
5. **Cheapness was never the broken axis.** The sub-floor ladder is half the control's wall time and produces an invalid partition. Any future arm should be judged on validity per unit compute, not on either alone.

## Related documents

- `docs/reference/winner_take_all_partition_gap.md` — the standing explanation; §4b states the resolution floor this report measures, §4c the locality criterion.
- `docs/plans/PHASE1_N1000_VALIDITY_PLAN.md` — P3 (mesh-budget validity floor) is the proposal this report supplies evidence for.
- `docs/experiments/05-soft-area-constraint/` — the arm the stall guard was written for, and where flooring genuinely was pathological.
- `docs/experiments/04-territory-aware-highn-validation/` — the other cheapening arm that failed on connectivity.
- Configs: `parameters/torus_100part_subfloor_ladder.yaml` (this run), `parameters/torus_300part_above_flo` (Stage 2, *not* run — gate failed).
- Code: `src/optimization/pgd_optimizer.py` (stall guard), `testing/watch_level_gates.py` (per-level gates).